

4 = Add 1 then 4

$$\text{numSum}(4) \quad \underbrace{1 + 2 + 3 + 4} = 10$$

$$\text{numSum}(3) + 4$$



$$\text{numSum}(2) + 3 + 4$$



$$\text{numSum}(1) + 2 + 3 + 4$$

$$\text{numSum}(5)$$



$$5 + \text{numSum}(4)$$



$$5 + 4 + \text{numSum}(3)$$

```
def mystery(n):
```

```
    if n <= 0:
```

```
        print("", end="")
```

```
    elif n % 2 == 0:
```

```
        print("(", end="")
```

```
        mystery(n - 1)
```

```
        print(")", end="")
```

```
    else:
```

```
        print("[", end="")
```

```
        mystery(n - 1)
```

```
        print("]", end="")
```

```
mystery(5)
```

mystery(5)

if 5 <= 0: X

elif 5 % 2 == 0? X

else

mystery(4)

mystery(4)

elif

mystery(3)

else:

mystery(2)

mystery(1)

else: mystery(0)

Output:

[[([*])]]]

$$3^4 = \underbrace{3 * 3 * 3 * 3}_{\leftarrow}$$

pow(3, 4)

3 * pow(3, 3)



def pow(base, exp): 3 * 3 * pow(3, 2)

if exp == 1:

return base 3 * 3 * 3 * pow(3, 1)

else:

base * pow(base, exp) 3 * 3 * 3 * 3

Fibonacci:

base cases 0, 1, 1, 2, 3, 5, 8, ...

Fibonacci(6)

↳ Fibonacci(5) + Fibonacci(4)

↳ Fib(4) + Fib(3)

↳ Fib(3) + Fib(2)

↳ Fib(2) + Fib(1)

↳ Fib(1) + Fib(0)

def fibonacci(num):

if n == 0:

return 0

if n == 1:

return 1

return fibonacci(num-1) + fibonacci(num-2)

fibonacci(6)

↳

fibonacci(5) + fibonacci(4)

↓

fib(4) + fib(3)

↓

fib(3) + fib(2)

⋮

Here 6 is the index of the Fibonacci number

Index n	0	1	2	3	4	5	6
Fibonacci	0	1	1	2	3	5	8

Fibonacci(6) gives Fibonacci number at the 6th index

Magic index problem: $my_array = [-40, -20, -1, 1, 2, 3, 5, 7, 9, 12, 13]$

```
def f(array, start, end):  
    if start > end:  
        return -1  
  
    mid = (start + end) // 2  
  
    if array[mid] == mid:  
        return mid  
    elif array[mid] > mid:  
        return f(array, start, mid - 1)  
    else:  
        return f(array, mid + 1, end)  
  
my_array = [-40, -20, -1, 1, 2, 3,  
            5, 7, 9, 12, 13]  
  
print(f(my_array, 0, len(my_array) - 1))
```

Output: 7

$f(array, 0, 10)$

start = 0

end = 10

mid = $(0 + 10) // 2$

$10 // 2 = 5$

array[5]

if $3 == 5$ x

elif $3 > 5$? x

else $3 < 5$ ✓

$\rightarrow f(array, 6, 10)$

if $6 > 10$?

mid = $(6 + 10) // 2$

$16 // 2 = 8$

array[8]

$9 == 8$ x

elif $9 > 8$? ✓

$\rightarrow f(array, 6, 7)$

if $6 > 7$? x

mid = $(6 + 7) // 2 = 13 // 2 = 6$

arr[6] = 5

if $5 == 6$? $\times$
elif $5 > 6$? $\times$
else

f(array, 7, 7)

7 returned here

if $7 > 7$? $\times$

mid = $(7 + 7) // 2 = 7$

arr[7] = 7

if $7 == 7$? $\checkmark$

↳ returns 7

we got to the stopping
point here!

permutation([B, E, V], 0, 2)
 if 0 == 2? x
 else: i in range(0, 3) = 0, 1, 2
 Outermost loop - iteration i=0
 start=0; end=2
 i: 0

Code tracing
 permutation

choose
 lst[0], lst[0] = lst[0], lst[0] ← list unchanged by swaps
 # exploration
 permutation(lst, 1, 2)

Output:

[B, E, V]
 [B, V, E]
 [E, B, V]
 [E, V, B]
 [V, E, B]
 [V, B, E]

start=1; end=2

if 1 == 2? x
 else:

i in range(1, 3): 1, 2

i=1

choose

list unchanged →

lst[1], lst[1] = lst[1], lst[1]

exploration start=2; end=2

permutation(lst, 2, 2)

if 2 == 2? ←

print!

return to caller

unchoose

list unchanged →

lst[1], lst[1] = lst[1], lst[1]

i=2

choose

lst[1], lst[2] = lst[2], lst[1]

exploration

permutation(lst, 2, 2)

Swaps 'E' and 'V'
 New list is

[B, V, E]

swap back 'V'
and 'E'

if $2 == 2?$ ✓
prints!

return to caller

Updated list is
['B', 'E', 'V']

unchoose

← $lst[1], lst[2] = lst[2], lst[1]$ Done w Iteration

unchoose

list not changed by
swap

← $lst[0], lst[0] = lst[0], lst[0]$

Outermost loop: iteration $i=1$
start (still) = 0; end = 2

$i=1$

choose

← $lst[0], lst[1] = lst[1], lst[0]$

exploration

permutation($lst, 0+1, 2$) — end
start

↳ if $1 == 2?$ ✗
- else

i in range(1, 3) : 1, 2

$i=1$

choose start: 1, end: 2

list remains same as
last step ['E', 'B', 'V']

← $lst[1], lst[1] = lst[1], lst[1]$

exploration

permutation($lst, 2, 2$)

↳ if $2 == 2?$ ✓
prints return

unchoose

list remains same
as last step ['E', 'B', 'V']

← $lst[1], lst[1] = lst[1], lst[1]$

$i=2$

choose

Swaps the list at
index 1 and 2 to get
['E', 'V', 'B']

← $lst[1], lst[2] = lst[2], lst[1]$

exploration

permutation($lst, 2, 2$)

↳ if $2 == 2?$ ✓
prints return

DONE W ITERATION

Swaps the list to get
['E', 'B', 'V']
Swaps the list to
get ['B', 'E', 'V']

unchoose

← $lst[1], lst[2] = lst[2], lst[1]$

unchoose

← $lst[0], lst[1] = lst[1], lst[0]$

Outermost loop: iteration 2

$i = 2$

start: 0 end: 2

choose

swaps the list to $\leftarrow \text{lst}[0], \text{lst}[2] = \text{lst}[2], \text{lst}[0]$

get ['V', 'E', 'B'] # exploration

permutation (lst, 1, 2)
start end
 $\rightarrow$ if $1 == 2?$ x

else

i in range(1, 3): 1, 2

$i = 1$ # choose

swaps keep list the same $\leftarrow \text{lst}[1], \text{lst}[1] = \text{lst}[1], \text{lst}[1]$
['V', 'E', 'B'] # exploration

permutation (lst, 2, 2)

$\rightarrow$ if $2 == 2?$ ✓ prints returns

unchoose

swaps keeps list the same $\leftarrow \text{lst}[1], \text{lst}[1] = \text{lst}[1], \text{lst}[1]$
['V', 'E', 'B']

$i = 2$

choose

swaps list to $\leftarrow \text{lst}[1], \text{lst}[2] = \text{lst}[2], \text{lst}[1]$
['V', 'B', 'E'] # exploration

permutation (lst, 2, 2)

$\rightarrow$ if $2 == 2?$ x prints! returns

DONE W ITERATION

swaps list to # unchoose

$\leftarrow \text{lst}[1], \text{lst}[2] = \text{lst}[2], \text{lst}[1]$
['V', 'E', 'B']

unchoose

swaps to ORIG list $\leftarrow \text{lst}[0], \text{lst}[2] = \text{lst}[2], \text{lst}[0]$
['B', 'E', 'V']

DONE! We generated all permutations of ['B', 'E', 'V']